Real-Time Recommendation System

Amazon Reviews 2023 (Books) — from raw data to a serving webapp

Presented by:
AIS 3

Project defense — RecSys course

DATA PREPARATION

Task for each member

Name	Task
Trần Duy Thái	Data Exporation, MF-BPR retrival and webapp
Thin Sapal Yu	Two tower retrival and webapp
Krishanth Dev	Baseline and webapp
Nguyễn Thái Dương	Ranking (feature engineering + LightGBM) and webapp (Bonus) Multi-source retrieval: blend candidates from MF-BPR + two-tower + popularity

DATA PREPARATION

Dataset: Amazon Reviews 2023 — Books

29.5M

raw interactions · 10.3M users · 4.45M items

5M

most recent kept (Jul 2020 → Sep 2023), per the 5M cap

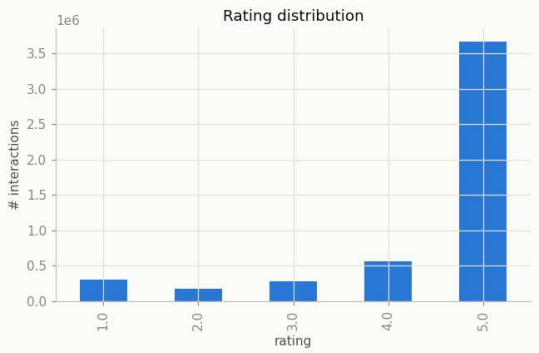
1.4e-6

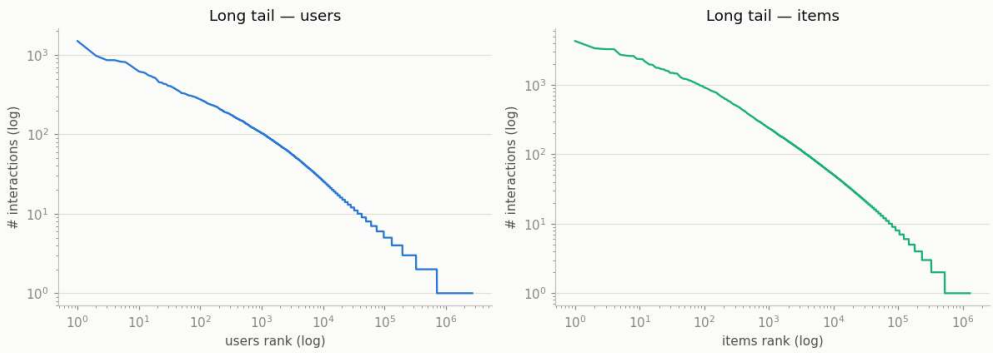
matrix density — far sparser than MovieLens

659K

interactions after k-core filtering (min 5 per user & item)

Design decisions. Subsample = most recent 5M (matches deployment: models serve on recent behavior). k-core (min 5/side) drops 86.8% of rows but removes one-shot users/items no CF model can learn — final: 49,235 users × 43,014 items. **Positive = any interaction** (a review means the user chose the book; rating kept as a feature).





73% of ratings are 5★
→ ratings carry little signal; interaction itself is the label.

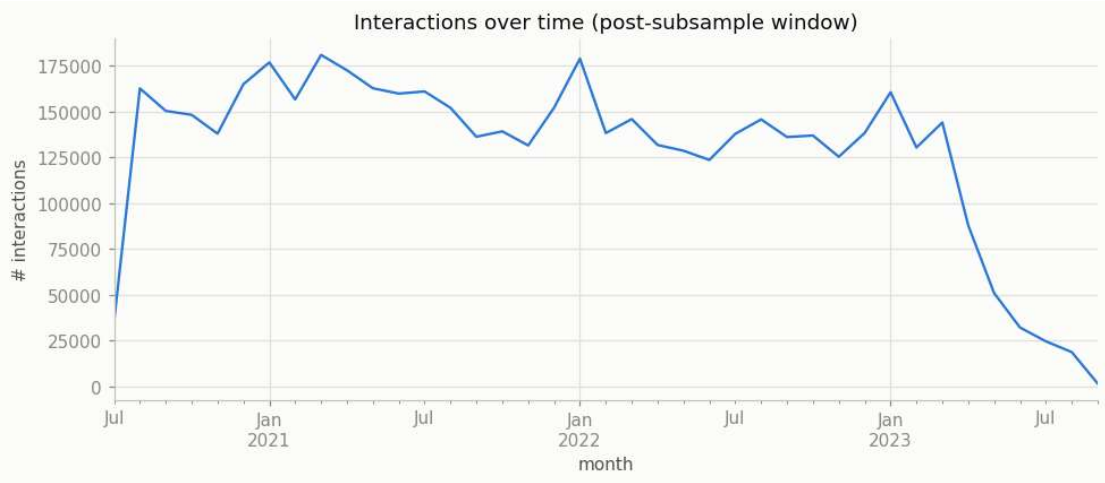
Classic power-law long tail
median user & item: 1 interaction → motivates k-core.

DATA PREPARATION

Time-based train / val / test split

Global timestamp cutoffs (90 / 5 / 5) — the model never sees the future. Validation used for all tuning; test touched once per model.

Split	Interactions	Users	Items	Date range
Train	592,954	48,199	40,366	2020-07-25 → 2023-01-18
Validation	18,656	10,179	8,446	2023-01-18 → 2023-03-05
Test	10,234	6,096	6,114	2023-03-05 → 2023-08-30



Why it matters

Rising interaction volume toward the window's end — a random split would leak future items and popularity trends into training and inflate every metric.

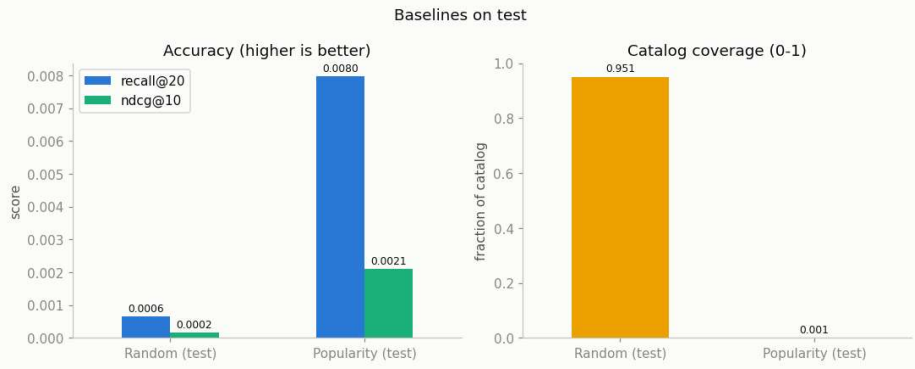
In the webapp, test-period interactions act as "recent user activity" the model was never trained on — simulating real deployment.

BASELINES

Baselines set the floor: Random & Popularity

Model	Split	Recall@20	NDCG@10	Coverage
Random	val	0.0006	0.0001	99.4%
Random	test	0.0006	0.0002	95.1%
Popularity	val	0.0096	0.0032	0.08%
Popularity	test	0.0080	0.0021	0.08%

Random — accuracy ≈ 0 on a 40K-item catalog, but near-perfect coverage. The two metrics pull in opposite directions.
Popularity — 13× Random on Recall@20, but everyone gets the same 20 bestsellers: coverage collapses to 32 items.
The bar to clear: a personalized model must beat Popularity on accuracy and coverage simultaneously.



Same evaluation harness (Recall@20/50, NDCG@10, catalog coverage) reused unchanged by every later stage — all results comparable.

RETRIEVAL

Retrieval 1 — MF-BPR, from scratch in PyTorch

Model. score(u, i) = user · item embedding + item bias. Trained with BPR pairwise loss: for each positive, sample a negative (popularity-weighted) and push σ(pos − neg) → 1.
Serving. Item bias folded into the FAISS inner-product index by appending bias as an extra dimension (user side gets a constant 1) → exact top-100 with IndexFlatIP.



Hyperparameter grid (val Recall@20)

dim	lr	Recall@20	NDCG@10	Coverage
64	**0.01**	0.0478	0.0161	0.54
32	0.01	0.0472	0.0157	0.52
64	0.05	0.0122	0.0032	0.66
32	0.05	0.0104	0.0031	0.64

lr matters far more than dim — 0.05 diverges. Final: dim=64, lr=0.01, 30 epochs.

0.051
Recall@20 (test) — 6.4× Popularity

0.016
NDCG@10 (test) — 7.7× Popularity

50%
catalog coverage (test)

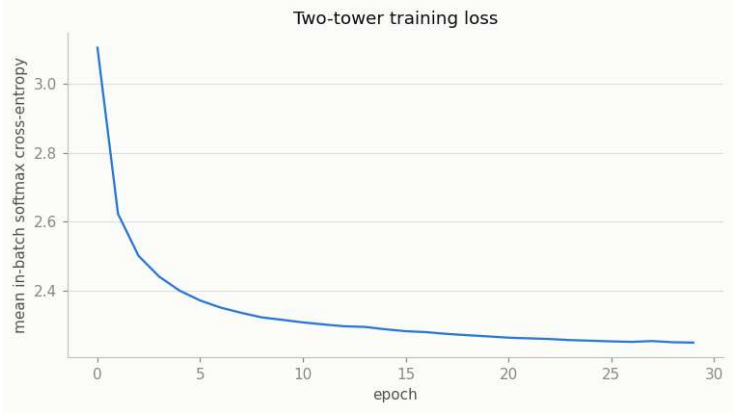
Validation: R@20 0.048 · R@50 0.097 · NDCG@10 0.017 · coverage 55% — val ≈ test: the model generalizes across the time gap.

RETRIEVAL

Retrieval 2 — Two-tower with in-batch negatives

User tower
mean-pooled MF-BPR history embedding
+ activity, avg rating, recency
→ MLP → 32-d

Item tower
item-id + category + author embeddings
+ 6 numeric features (rating, price, popularity, recency…)
→ MLP → 32-d



Training: batch of B (user, positive) pairs → B×B score matrix → softmax cross-entropy; the other B−1 items are free in-batch negatives.

Best config: dim=32, lr=0.01 (0.05 collapses — coverage 2%). Cold-start friendly: item tower can embed a new book from metadata alone.

0.035
Recall@20 (test)

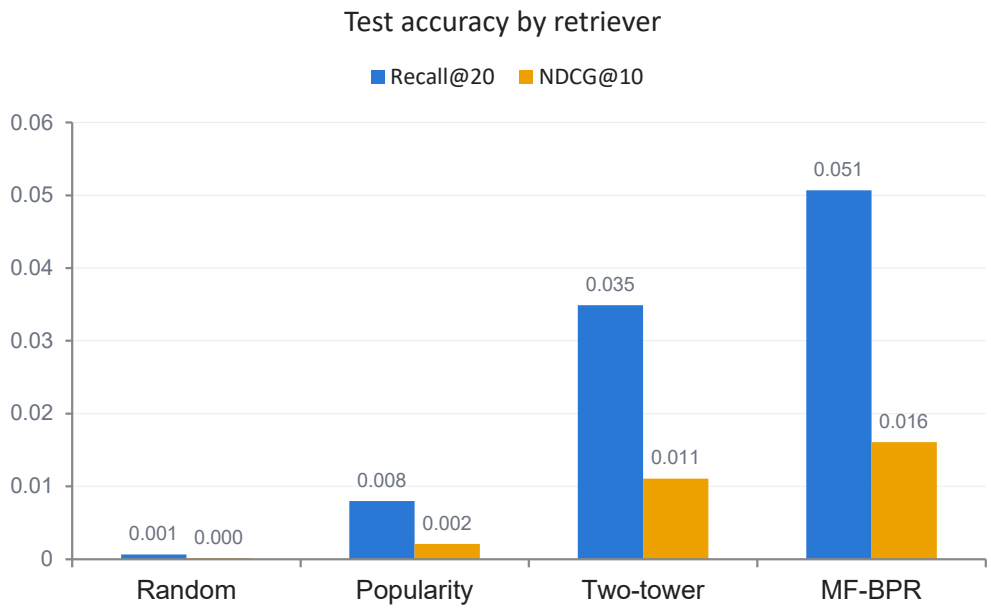
0.011
NDCG@10 (test)

94%
catalog coverage (test)

Validation: R@20 0.049 · R@50 0.089 · NDCG@10 0.016 · coverage 97% — matches MF-BPR on val, but degrades on the later test window (temporal drift).

RETRIEVAL

Retrieval comparison: accuracy vs coverage trade-off



Model	Split	R@20	R@50	NDCG@10	Cov.
Random	val	0.0006	—	0.0001	99.4%
Random	test	0.0006	—	0.0002	95.1%
Popularity	val	0.0096	—	0.0032	0.08%
Popularity	test	0.0080	—	0.0021	0.08%
MF-BPR	val	0.0484	0.0968	0.0165	54.7%
MF-BPR	**test**	**0.0507**	**0.0954**	**0.0161**	**50.5 %**
Two-tower	val	0.0490	0.0886	0.0165	97.5%
Two-tower	test	0.0349	0.0708	0.0111	94.5%

MF-BPR wins on pure accuracy (memorizes collaborative signal best), and holds up from val to test.

Two-tower ties MF-BPR on val (R@20 0.049) but drops on test — more sensitive to temporal drift — while keeping ~95% coverage. Both feed the ranker next.

RANKING

Ranking stage: 14 features across 4 required groups

USER (4)

- activity count (log)
- avg rating given
- days since last interaction
- last interaction hour

ITEM (5)

- popularity (log)
- avg rating
- age in days
- price (normalized)
- category (categorical)

CROSS (3)

- author affinity (recency-decayed)
- user–category affinity
- category-overlap affinity

RETRIEVAL (2)

- retrieval rank
- retrieval score

Training setup. LightGBM LambdaRank, one ranker per retriever (Popularity / MF-BPR / Two-tower). Top-100 candidates per train user → graded labels: 2 = interacted & rating ≥ 4, 1 = interacted, 0 = not. All aggregates computed on train only, with NOW = train max timestamp — no leakage. Early stopping on honest val NDCG@10 (users with no relevant candidate excluded — LightGBM's built-in NDCG counts them as perfect).

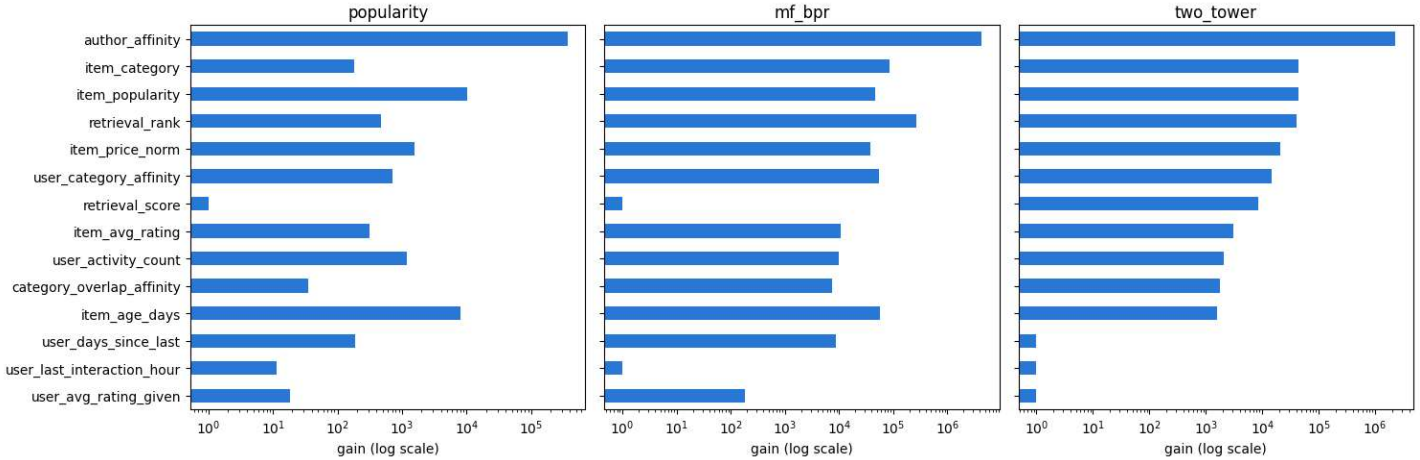
RANKING

Ablation: LightGBM re-ranking lifts every retriever

retriever	split	model	stage	ndcg@10	recall@20	recall@50	catalog_coverage
mf_bpr	test	lightgbm	ranking	0.136052	0.382707	0.667700	0.286814
		retrieval_only	ranking	0.095768	0.327690	0.619575	0.211420
	val	lightgbm	ranking	0.170543	0.454090	0.673213	0.329451
		retrieval_only	ranking	0.094546	0.308208	0.627970	0.241503
popularity	test	lightgbm	ranking	0.123751	0.274045	0.567852	0.002581
		retrieval_only	ranking	0.050469	0.210474	0.500988	0.000744
	val	lightgbm	ranking	0.132527	0.350368	0.616943	0.002697
		retrieval_only	ranking	0.066205	0.222529	0.465316	0.000744
two_tower	test	lightgbm	ranking	0.163908	0.441609	0.691116	0.503673
		retrieval_only	ranking	0.087430	0.310722	0.627963	0.512205
	val	lightgbm	ranking	0.202215	0.519039	0.735210	0.583275
		retrieval_only	ranking	0.102918	0.349833	0.647015	0.589343

ANALYSIS

Feature importance: author affinity dominates



author_affinity is #1 for all three rankers

"You read this author before" is the strongest signal in book recommendation — robust across retrievers.

retrieval_rank matters most for MF-BPR

The ranker trusts the strongest retriever's ordering.

A feature that stays important regardless of which retriever fed it is genuinely informative. Cross features (author & category affinity) + item popularity/age form the stable core; pure user features (avg rating given, interaction hour) contribute little — consistent with the very short median history per user.

RANKING

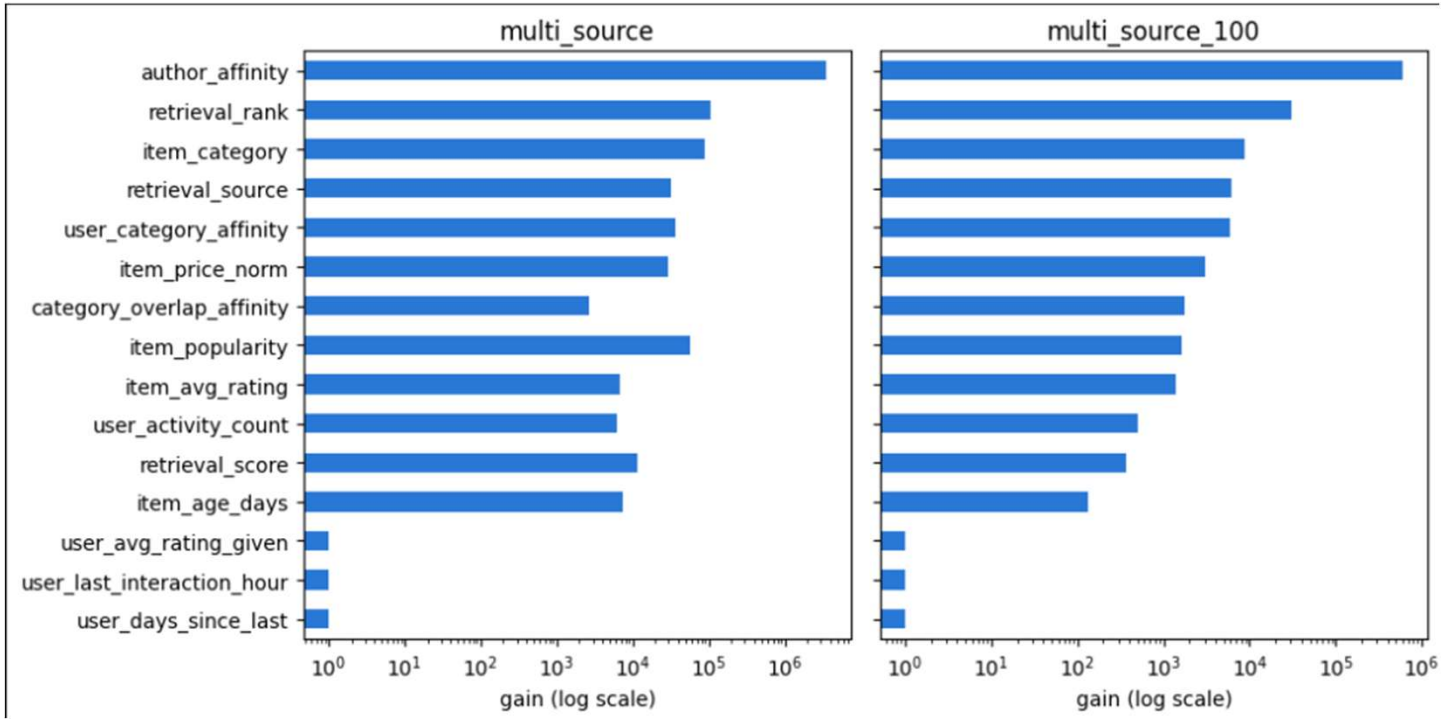
Combine retriever

We get 60 candidates from Two-tower, 30 from MF-BPR and 10 from Popularity

			stage	ndcg@10	recall@20	recall@50	catalog_coverage
retriever	split	model					
mf_bpr	test	lightgbm	ranking	0.136052	0.382707	0.667700	0.286814
		retrieval_only	ranking	0.095768	0.327690	0.619575	0.211420
	val	lightgbm	ranking	0.170543	0.454090	0.673213	0.329451
		retrieval_only	ranking	0.094546	0.308208	0.627970	0.241503
multi_source	test	lightgbm	ranking	0.152345	0.423154	0.714458	0.473660
		retrieval_only	ranking	0.088617	0.301765	0.618376	0.227438
	val	lightgbm	ranking	0.190268	0.508924	0.761400	0.553657
		retrieval_only	ranking	0.069533	0.231412	0.619624	0.258753
multi_source_100	test	lightgbm	ranking	0.153301	0.433405	0.712504	0.409634
		retrieval_only	ranking	0.100498	0.344669	0.639625	0.229414
	val	lightgbm	ranking	0.177084	0.476507	0.718684	0.483656
		retrieval_only	ranking	0.085736	0.284362	0.621387	0.260008
popularity	test	lightgbm	ranking	0.123751	0.274045	0.567852	0.002581
		retrieval_only	ranking	0.050469	0.210474	0.500988	0.000744
	val	lightgbm	ranking	0.132527	0.350368	0.616943	0.002697
		retrieval_only	ranking	0.066205	0.222529	0.465316	0.000744
two_tower	test	lightgbm	ranking	0.163908	0.441609	0.691116	0.503673
		retrieval_only	ranking	0.087430	0.310722	0.627963	0.512205
	val	lightgbm	ranking	0.202215	0.519039	0.735210	0.583275
		retrieval_only	ranking	0.102918	0.349833	0.647015	0.589343

ANALYSIS

Feature importance: author affinity dominates



author_affinity is still #1

"You read this author before" is the strongest signal in book recommendation — robust across retrievers.

SERVING

Demo webapp: each page uses a different model

Page / section	Model	Behavior
Homepage — not logged in	Popularity	Trending / most popular books
Homepage — logged in	Two-tower → LightGBM	Personalized top-10, re-ranked in real time
Item page — "Similar items"	Cosine on item embeddings	Nearest neighbors of the current book
"Because you liked X" row	Same embeddings	Neighbors of one book from the user's history

PostgreSQL
user profiles, interaction history, item metadata (title, image, category)

FastAPI
loads models + FAISS at startup; REST endpoints with Swagger at /docs

Streamlit
calls the API; shows which model produced each recommendation

Docker Compose
postgres · api · frontend — one command: docker compose up

Recommendations vary on reload (temperature sampling over top candidates) · unknown users fall back to Popularity · item-to-item similarities precomputed

CONCLUSION

Conclusion: what we built, what we learned

2

retrievers implemented from scratch in PyTorch (MF-BPR, two-tower)

0.164

NDCG@10 of the best pipeline: two-tower → LightGBM (test)

+87%

NDCG@10 gained by re-ranking two-tower candidates

4

services in one command: postgres · api · frontend · docker compose up

1 **Ranking is where the gains are**
Swapping retrievers moved NDCG@10 by ~0.01; adding the LightGBM ranker nearly doubled it. Candidate generation gets you in the room — re-ranking wins the top-10.

2 **Coverage decides the final winner**
MF-BPR retrieves most accurately, but the two-tower's 94% catalog coverage hands the ranker richer candidates — so the "weaker" retriever produces the strongest end-to-end system.

3 **Simple domain signals beat learned ones**
One recency-weighted "has this user read this author?" feature out-gains every embedding-derived feature, for every retriever. Cheap, interpretable, robust.

Main limitation — sparsity & drift. Median user has 1 interaction pre-filter, and the two-tower loses accuracy on the later test window. With more time: a sequential retriever (SASRec) to exploit order, hard-negative mining, and MMR re-ranking for explicit relevance–diversity control.

Thank you.